

CSE 554: Systems for Machine Learning– Assignment 3

In this assignment, you will profile the performance of operations used in transformers and the transformer as a whole. The assignment consists of three parts: GEMM profiling, Attention Profiling, and Transformer Profiling. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the following questions (.pdf)** and the **code**.

- **Note:** When compiling libraries, please use make with “-j 32” rather than “-j” only. This would limit CPU usage and avoid crashing making the machine unresponsive.
- For both matrix multiplication and attention, we use FP16 data type.

Section 1: General Matrix Multiplication

Q1. 2 + 5 + 3 points

- What is the operational intensity of a GEMM in terms of M, N, K dimensions?

For the GEMM $C = AB$ where A has shape (M, K), B has shape (K, N), and C has shape (M, N), the FLOPs is $2MNK$ (multiplication plus addition). When using FP16, the number of bytes read or written is $2(MK+KN+MN)$ (2 bytes per element; A and B are read once, C is written once). The operational intensity is thus: $2MNK / (2(MK+KN+MN)) = MNK / (MK + KN + MN)$.

- Calculate the compute intensity for M in 128 - 2048 with step 128, (N, K) = (512, 512), (4096, 4096), (14336, 4096), (4096, 1024), (1024, 4096) and show the result in a table. You can use <https://www.tablesgenerator.com> to generate tables.

M / (N, K)	(512, 512)	(4096, 4096)	(14336, 4096)	(4096, 1024)	(1024, 4096)
128	85.33	120.47	123.06	110.70	110.70
256	128.00	227.56	236.96	195.05	195.05
384	153.60	323.37	342.69	261.45	261.45
512	170.67	409.60	441.11	315.08	315.08
640	182.86	487.62	532.94	359.30	359.30
768	192.00	558.55	618.82	396.39	396.39
896	199.11	623.30	699.32	427.94	427.94
1024	204.80	682.67	774.92	455.11	455.11

1152	209.45	737.28	846.06	478.75	478.75
1280	213.33	787.69	913.12	499.51	499.51
1408	216.62	834.37	976.45	517.89	517.89
1536	219.43	877.71	1036.34	534.26	534.26
1664	221.87	918.07	1093.07	548.95	548.95
1792	224.00	955.73	1146.88	562.20	562.20
1920	225.88	990.97	1197.99	574.21	574.21
2048	227.56	1024.00	1246.61	585.14	585.14

- What is CUTLASS? What is CUBLAS? What is the key difference between them?
 - <https://docs.nvidia.com/cuda/cublas/>
 - <https://github.com/NVIDIA/cutlass>

cuBLAS is NVIDIA's official GPU BLAS (Basic Linear Algebra Subprograms) library. It has highly optimized implementations of BLAS routines like GEMM.

CUTLASS is an open-source CUDA C++ template library that contains “a collection of abstractions for implementing high-performance matrix-matrix multiplication (GEMM) and related computations at all levels and scales within CUDA.”

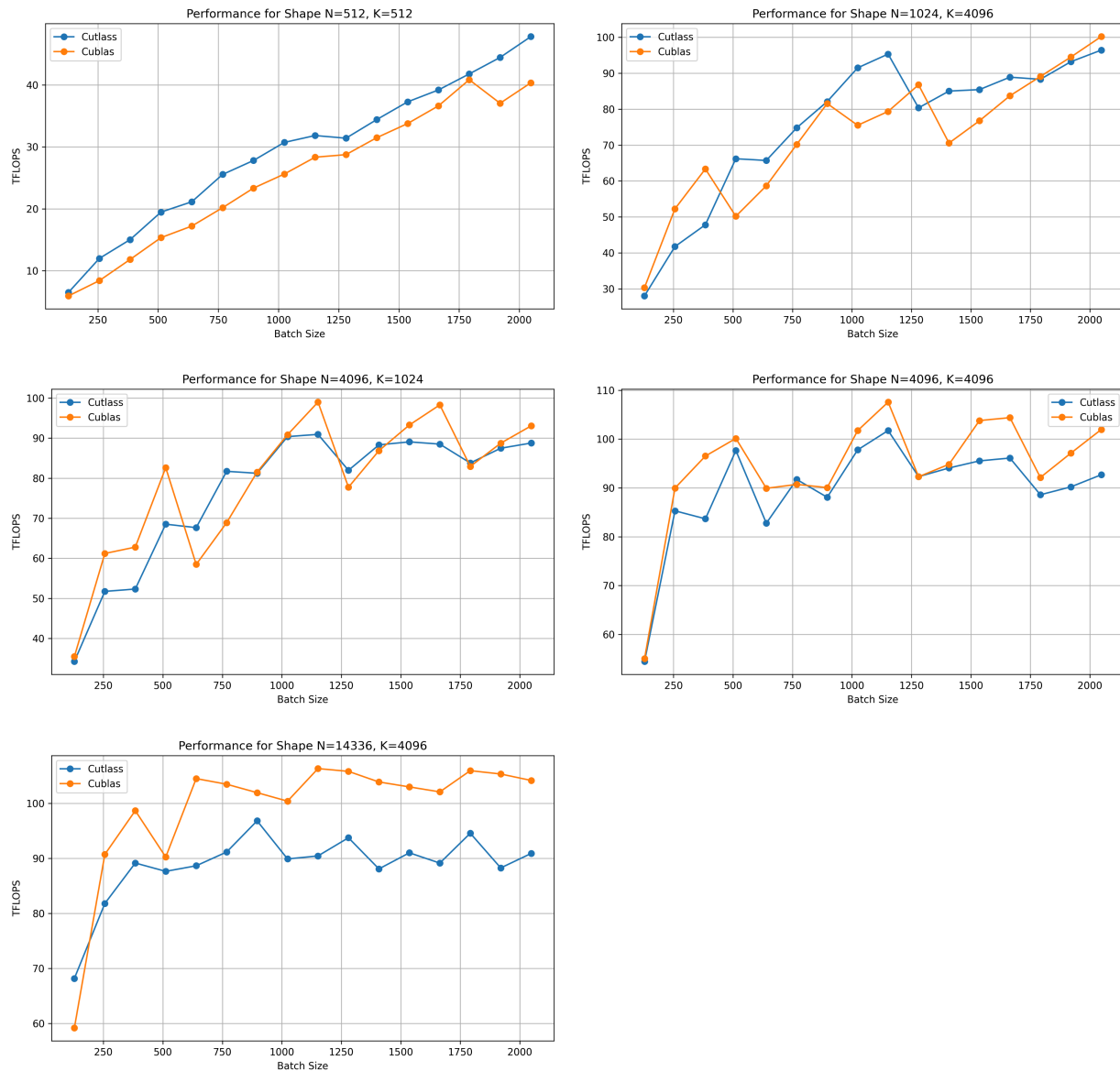
Key difference: cuBLAS is a pre-compiled, black-box, ready-to-call library, while CUTLASS is a kernel construction toolkit that allows customization of GEMM kernels.

Q2. 5 + 5 + 3 + 2 points

- Profile the CUBLAS cublasGEMMEx performance for the above shapes and plot the performance. Plot one figure for one shape with the M dimension on the x-axis and TFLOPs (total compute/time) on the y-axis.
 - Please use the profiling methods in [Section1/prof_example.cu](#). We do warmups and L2 cache flush to increase the profiling accuracy.
 - The example plotting script is given at [Section1/plot_gemm.py](#).
- Profile the CUTLASS GEMM performance using the CUTLASS profiler for the above shapes. The cutlass profiler will iterate through all kernel implementations with different tile sizes and configurations. Use again the `--profiling-iterations` of 100. Profile different `--split_k_slices=1,2,4,8` with `--split_k_mode=serial`. Split K will control the number of splits in K dimension. For example, for GEMM shape (128, 1024, 4096), when split K is 4, the kernel will first treat it as 4 GEMM problems of size (128, 1024, 1024) and perform a reduction to add the partial results. Take the best performance across all implementations for each GEMM shape. Plot the CUTLASS performance in the same figure as CUBLAS.
 - <https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/quickstart.md>

- `$ cmake .. -DCMAKE_CUDA_ARCHITECTURES=75`
`-DCUTLASS_ENABLE_TESTS=OFF -DCUTLASS_ENABLE_EXAMPLES=OFF`
`-DCUTLASS_NVCC_ARCHS=75`
- `$ cmake --build . -j 32`

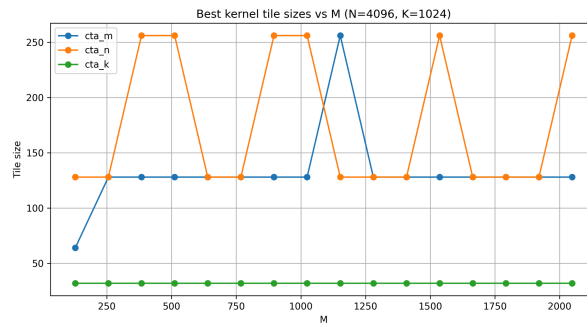
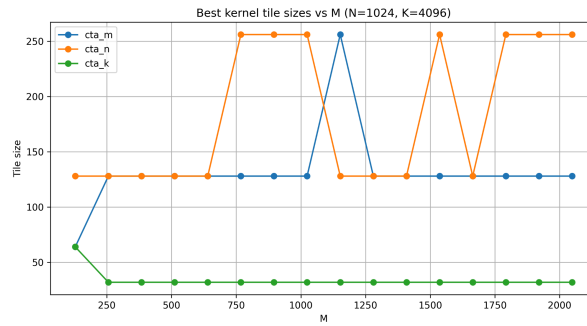
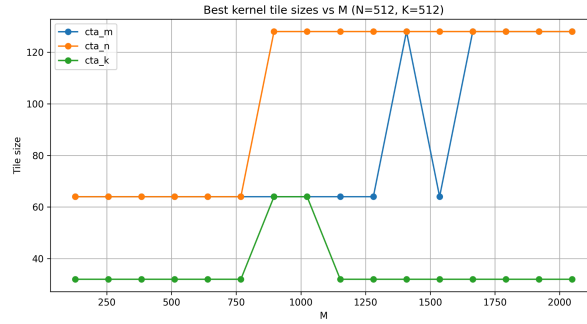
Profiling results of the shapes in cuBLAS and CUTLASS libraries:

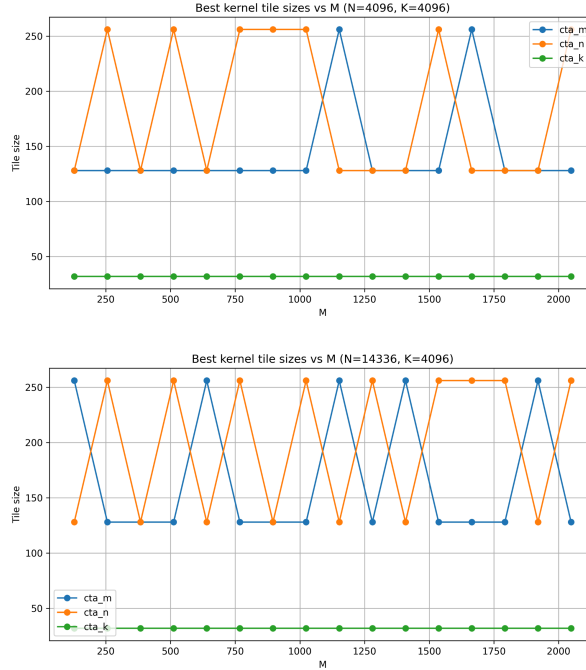


- Investigate the kernel that achieves the best performance in CUTLASS. What is the relationship between problem size, tile size, and split K?
 - https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/gemm_api_3x.md
- 1. The highest-performing kernels predominantly use `cta_k = 32`. The primary exceptions occur for the smaller or moderately sized problems (e.g., $N, K = 512 \times 512$ and 1024×4096), where `cta_k = 64` occasionally yields better performance.

For the spatial tile dimensions, cta_m and cta_n are typically 128 or 256, except when the problem size itself is small (e.g., $M, N = 512 \times 512$), in which case smaller CTA tiles are selected.

In all cases, the tile sizes (cta_m, cta_n, cta_k) are smaller than the corresponding problem sizes (M, N, K), as expected for a tiled GEMM decomposition.



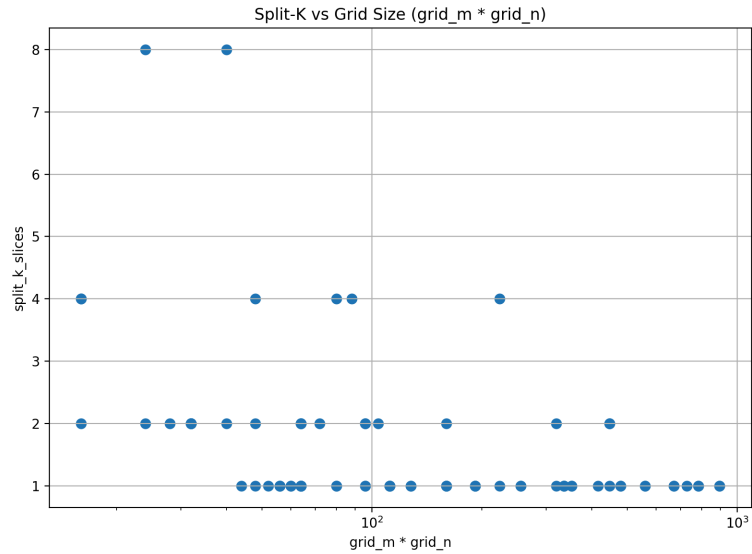


2. Define $\text{grid_m} = M / \text{cta_m}$, $\text{grid_n} = N / \text{cta_n}$.

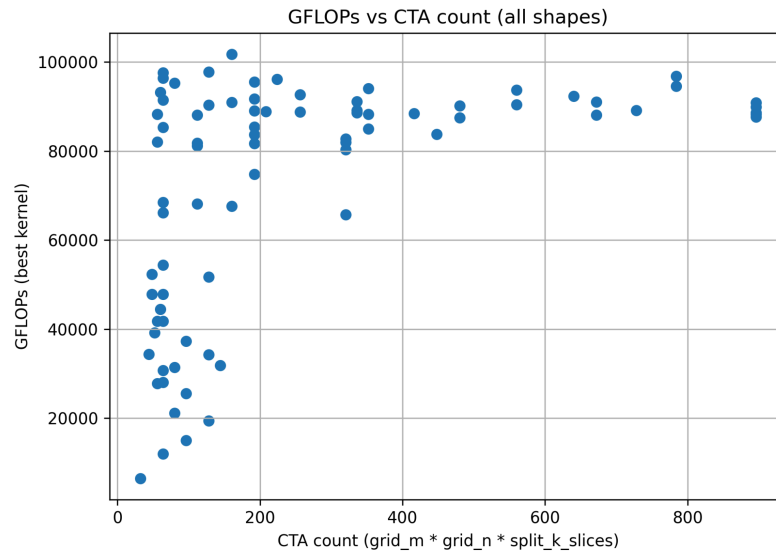
The number of CTAs launched in parallel is $\text{grid_m} * \text{grid_n} * \text{split_K_slices}$.

We observe an inverse relationship between tiling at M and N dimension and the use of split K. When $\text{grid_m} \times \text{grid_n}$ is small (i.e. tile sizes at M and N dimensions are large), the number of spatial CTAs is limited. In this case, increasing split_K_slices (i.e., introducing parallelism along the K dimension) helps increase the total CTA count and improves hardware utilization. When $\text{grid_m} \times \text{grid_n}$ is large, spatial tiling alone already generates sufficient CTAs to saturate the GPU. Under these conditions, $\text{split_K_slices} = 1$ is typically optimal, since additional K-partitioning would introduce unnecessary reduction overhead.

In other words, Split-K acts as a compensatory parallelization mechanism: it is primarily activated when spatial parallelism is insufficient to fully occupy the machine.



3. GFLOP/s throughput saturates once the number of concurrently executing CTAs exceeds a certain threshold. This plateau indicates that the GPU's compute resources are nearly fully utilized, and additional parallelism no longer translates into higher performance.



- What is the difference between the CUTLASS performance and the CUBLAS performance for different shapes? Describe your findings in a few sentences.
 1. For the smaller shape of $N, K=512, 512$, CUTLASS has better performance than cuBLAS.
 2. For the medium shapes where $(N, K) = (1024, 4096)$ or $(4096, 1024)$, CUTLASS's performance is quite similar to that of cuBLAS. For these shapes, sometimes CUTLASS performs a bit better, and other times cuBLAS is a bit better.
 3. For the larger shapes, cuBLAS consistently achieves higher TFLOPs than CUTLASS. For $N=14336, K=4096$, CUTLASS saturates at a peak performance of around 90

TFLOPS, while cuBLAS achieves over 100 TFLOPs. For $N=4096$, $K=4096$, cuBLAS performs slightly better than CUTLASS for most values of M . Overall, cuBLAS and CUTLASS have similar performance, and CUTLASS is slightly better for smaller shapes and cuBLAS is slightly better for larger shapes.

Section 2: Attention

Q1. 5 + 5 points

- Given model num_kv_heads , num_qo_heads , head_dim , what is the operational intensity of
 - Prefill attention, when the prompt length is p
 - Number of FLOPs:
 - Decode attention, when the context length is c

Attention FLOPs are dominated by the large matrix multiplications QK^T and AV , so we only count those in our calculation. For prefill attention, the FLOPs are roughly $4 * \text{num_qo_heads} * p^2 * \text{head_dim}$. Assuming each of Q , K , V is read once and the output is written once, we can count the number of bytes accessed for prefill attention (when using FP16): $4 * p * \text{head_dim} * (\text{num_qo_heads} + \text{num_kv_heads})$. The operational intensity is: $\text{num_qo_heads} * p / (\text{num_qo_heads} + \text{num_kv_heads})$.

For decode attention, the FLOPs are roughly $4 * \text{num_qo_heads} * c * \text{head_dim}$. The number of bytes accessed is: $4 * \text{head_dim} * (\text{num_qo_heads} + \text{num_kv_heads} * c)$. The operational intensity is: $\text{num_qo_heads} * c / (\text{num_qo_heads} + \text{num_kv_heads} * c)$.

- Compute the operational intensity for llama3-1B, llama3-3B, llama3-8B
 - For prefill attention, $p = 2^7, 2^8, \dots, 2^{15}$
 - For decode attention, $c = 2^7, 2^8, \dots, 2^{15}$

The configurations are:

- llama3-1B: $\text{hidden_size} = 2048$, $\text{num_qo_heads} = 32$, $\text{num_kv_heads} = 8$, $\text{head_dim} = 64$.
- llama3-3B: $\text{hidden_size} = 3072$, $\text{num_qo_heads} = 24$, $\text{num_kv_heads} = 8$, $\text{head_dim} = 128$.
- llama3-8B: $\text{hidden_size} = 4096$, $\text{num_qo_heads} = 32$, $\text{num_kv_heads} = 8$, $\text{head_dim} = 128$.

For prefill attention:

p	1B	3B	8B
-----	----	----	----

2 ⁷	102.40	96.00	102.40
2 ⁸	204.80	192.00	204.80
2 ⁹	409.60	384.00	409.60
2 ¹⁰	819.20	768.00	819.20
2 ¹¹	1638.40	1536.00	1638.40
2 ¹²	3276.80	3072.00	3276.80
2 ¹³	6553.60	6144.00	6553.60
2 ¹⁴	13107.20	12288.00	13107.20
2 ¹⁵	26214.40	24576.00	26214.40

For decode attention:

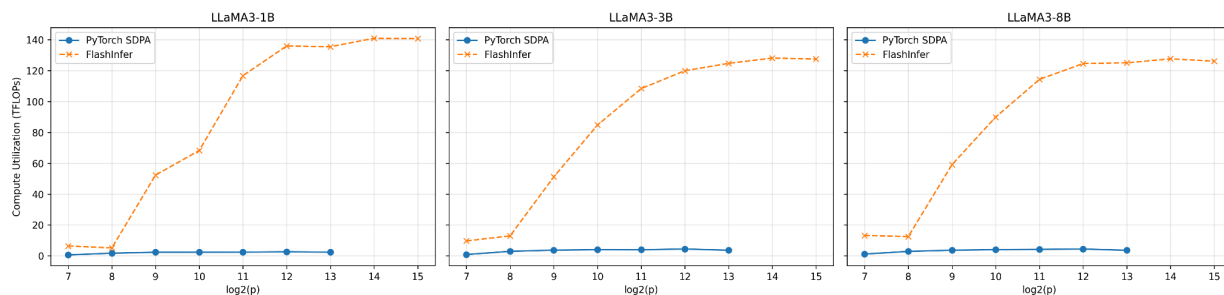
c	1B	3B	8B
2 ⁷	3.88	2.93	3.88
2 ⁸	3.94	2.97	3.94
2 ⁹	3.97	2.98	3.97
2 ¹⁰	3.98	2.99	3.98
2 ¹¹	3.99	3.00	3.99
2 ¹²	4.00	3.00	4.00
2 ¹³	4.00	3.00	4.00
2 ¹⁴	4.00	3.00	4.00
2 ¹⁵	4.00	3.00	4.00

Q2. 10 + 10 points

- Install Flashinfer using `uv pip install flashinfer-python==0.5.3`
- Evaluate the prefill attention performance of one layer using `torch.nn.functional.scaled_dot_product_attention` and `FlashInfer` for
 - Batch size = 1, $p = 2^7, 2^8, \dots, 2^{15}$. Plot the compute utilization TFlops of each model. Use $\log_2(p)$ as the x-axis. Generate one **subplot per model** inside a shared figure. In each subplot, draw line plots for both torch SDPA and FlashInfer using different line styles or colors. Label axes clearly. (see `Section2/plot_attention_p.py`)

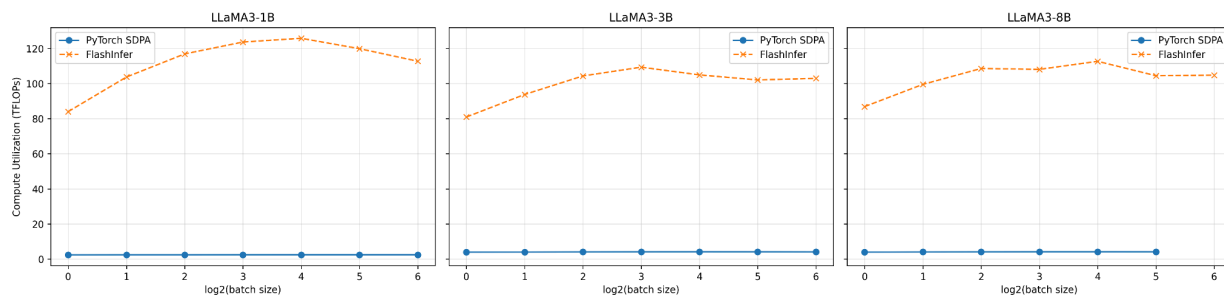
Note: For PyTorch SDPA, some data points are missing because of OOM errors.

Prefill Attention Compute Utilization vs Prompt Length (Batch=1)



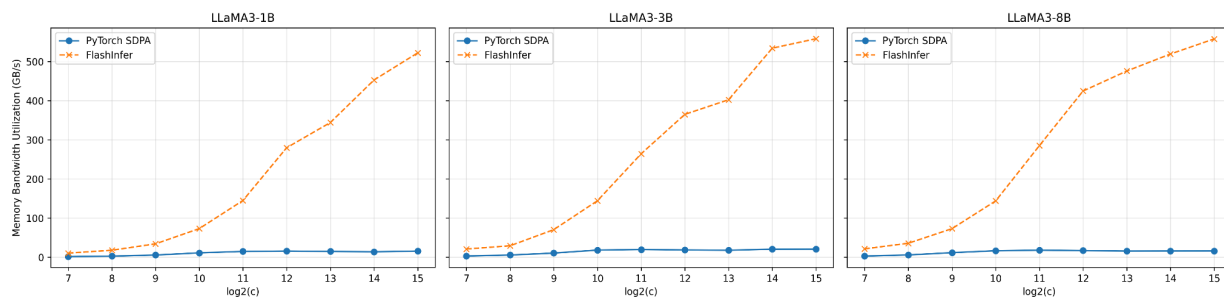
- Batch size = $2^0, 2^1, \dots, 2^6$, $p = 1024$ for all three models. Plot compute utilization using log batch size as the x-axis.

Prefill Attention Compute Utilization vs Batch Size ($p=1024$)



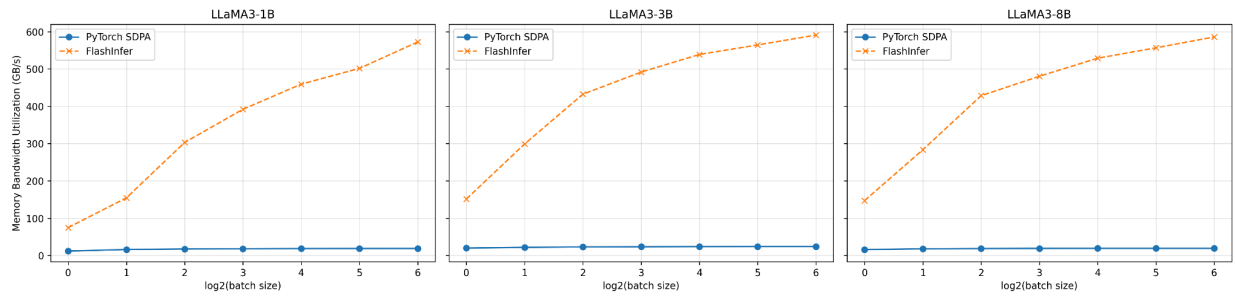
- Evaluate the decode attention performance of one layer using torch SDPA and FlashInfer
 - Batch size = 1, $c = 2^7, 2^8, \dots, 2^{15}$. Plot memory bandwidth utilization (GB/s) similar to previous questions.

Decode Attention Memory Bandwidth vs Context Length (Batch=1)



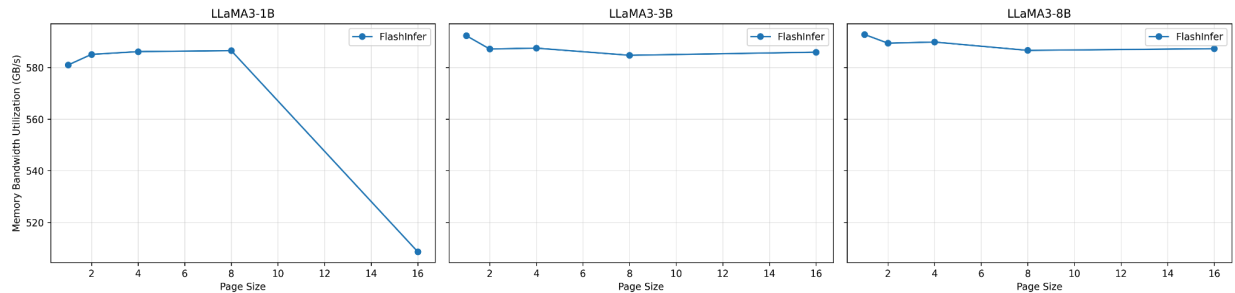
- Batch size = $2^0, 2^1, \dots, 2^6$, $c = 1024$ for all three models. Plot memory bandwidth utilization similarly.

Decode Attention Memory Bandwidth vs Batch Size (c=1024)



- Batch size = 128, c = 1024, page size = 1, 2, 4, 8, 16 for all three models using FlashInfer. Plot the memory bandwidth utilization of each model.

Decode Attention (FlashInfer) Bandwidth vs Page Size (Batch=128, c=1024)



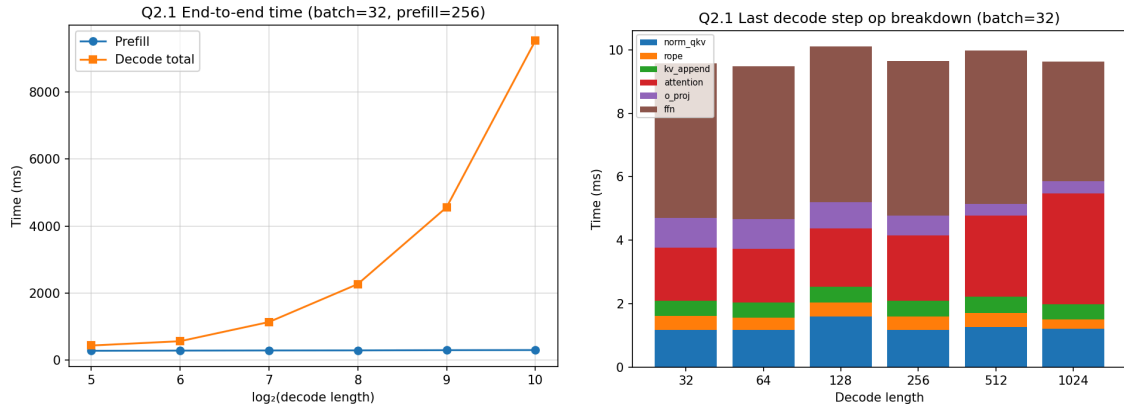
Section 3: End-to-end performance

Q1. 15 points

- Starting from the [Section3/flashinfer_pipeline.py](#), construct a serving engine using FlashInfer attention and rope kernels with paged attention.

Q2. 10 + 10 + 10 points

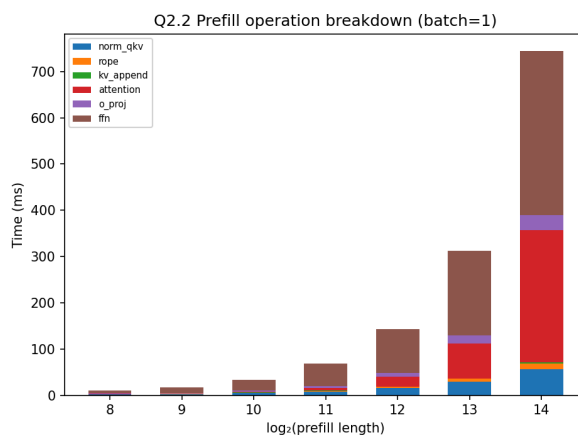
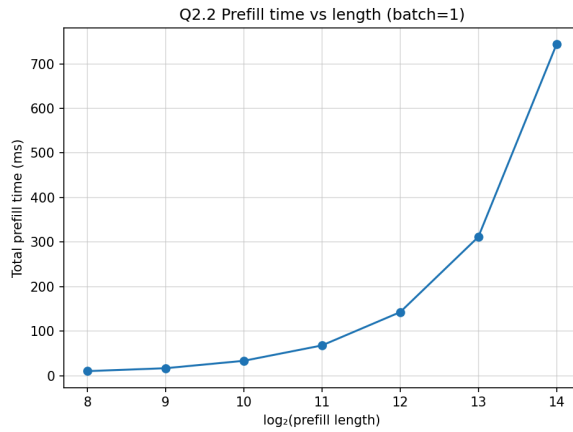
- With batch size 32, prefill length 256, decode length $2^5 - 2^{10}$, profile the prefill time and total decode time. Plot end-to-end time curve with the $\log(\text{decode length})$ as the x-axis. Which phase is the key bottleneck? In the last decode cycle, which operation takes the longest time for different decode lengths?



Clearly the decode phase is the bottleneck. The prefill time is stable as we increase the decode length, but the decode time keeps increasing. As a result, the decode time becomes way larger than the prefill time.

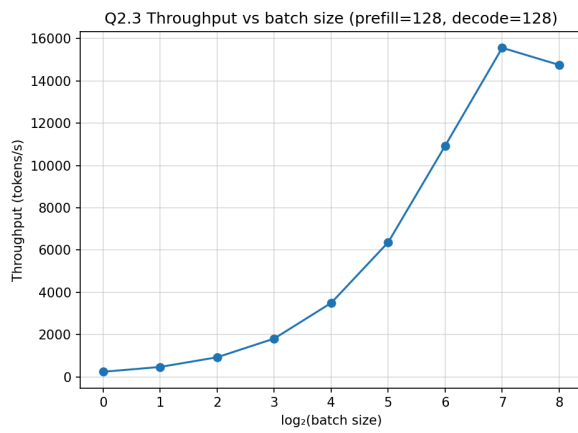
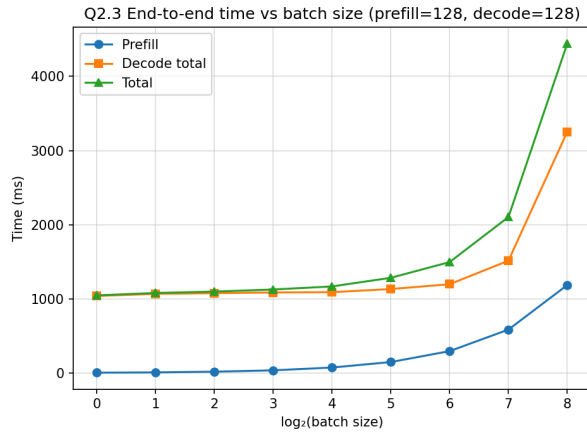
In the last decode cycle, FFN takes the longest time for all decode lengths. However, as the decode length increases, the proportion of the time taken by attention keeps increasing. When decode length is 1024, the attention time becomes comparable to the FFN time.

- With batch size 1, prefill length $2^8 - 2^{14}$, profile the prefill time. Plot end-to-end time curve with $\log(\text{prefill length})$ as x-axis. Examine the prefill time breakdown. What operations are the dominating factor for various prefill lengths?



For all prefill lengths, FFN time is the most dominant factor. But as the prefill length increases, attention time's proportion increases rapidly. For prefill lengths $\geq 2^{12}$, attention time is the second most dominant factor. Other operations take up a relatively small amount of time.

- With batch size $2^0 - 2^8$, prefill length 128, decode length 128, plot the end-to-end time curve with $\log(\text{batch size})$ as x-axis. Plot the total throughput $((\text{prefill} + \text{decode}) / \text{time})$ curve with $\log(\text{batch size})$ as x-axis. When does the performance saturate?



The performance saturates around the batch size of 2^7 . When batch size $< 2^7$, throughput increases as batch size increases. After this point, we observe throughput to drop a bit when batch size is 2^8 .